

Supplementary Material: Time-of-Use Pricing for Fixed-Capacity Inference Services

Anonymous Author

S1. Parameters

Table 1 reports the main parameters used in the peak-shaving experiments. Unless stated otherwise, values come from `pricing_sim/peak_shaving_config.py`. These are synthetic calibrations for mechanism analysis, not production estimates.

Table 1: Main simulation parameters

Parameter	Value	Meaning
T	8	Number of periods in one day.
h	3 hours	Length of one period.
G_A, G_B baseline	1500, 600	Fixed provider capacities.
G_A, G_B congested	300, 120	Tightened capacities used to activate QoS degradation.
π_R, π_E baseline	0.6, 0.4	Population shares of time-rigid and time-flexible users.
π_R, π_E congested	0.4, 0.6	Population shares in the congested experiments.
<code>base_rigid</code>	200,150,300,600, 800,900,700,250	Rigid-demand baseline.
<code>time_preference</code>	0.30,0.20,0.50,0.80, 1.00,1.00,0.90,0.40	Exogenous period preference.
<code>intermediary_brand</code>	1.05	Intermediary convenience parameter.
<code>firm_brand</code>	1.0, 1.0	Providers are nearly homogeneous in model quality.
<code>flexible_baseline</code>	400	Flexible-demand scale.
<code>rigid_wtp</code>	1.80	Reservation value for rigid or replay demand.
<code>rigid_churn_rate</code>	5.0	Price-response slope for rigid demand.
<code>market_growth</code>	1.2	Flexible market expansion at low prices.
p_0	0.80	Baseline price.
Direct price bounds	[0.45, 2.10]	Provider direct retail-price bounds.
Wholesale price bounds	[0.25, 0.90]	Provider wholesale-price bounds.
α_R, α_E	2.0, 5.0	Price sensitivity of time-rigid and time-flexible users.
c_s^R, c_s^E	0.60, 0.10	Switching costs of time-rigid and time-flexible users.
$\bar{u}$	0.82	QoS degradation threshold.
<code>qos_strength</code>	15.0	QoS degradation strength.

Parameter	Value	Meaning
γ	1.0	QoS feedback weight in utility.
c_g	0.015	Fixed capacity holding cost.
c_q	0.35	QoS degradation cost.
u_0	0.0	Utility of the no-purchase outside option.

S2. Pricing Shape and Grids

Provider strategies are restricted to four scalar parameters, $(\bar{w}_m, \delta_m, \bar{p}_m^D, \delta_m^D)$. The period prices are

$$w_{m,t} = \bar{w}_m(1 + \delta_m \hat{\ell}_t), \quad p_{m,t}^D = \bar{p}_m^D(1 + \delta_m^D \hat{\ell}_t),$$

with clipping to the allowed price bounds. Uniform pricing is implemented by setting $\hat{\ell}_t = 0$, so internal δ values do not create period-varying prices.

The coarse fictitious-play grid uses

$$\begin{aligned} \bar{w} &\in \{0.30, 0.85\}, & \delta &\in \{-0.4, -0.133, 0.133, 0.4\}, \\ \bar{p}^D &\in \{0.70, 1.50\}, & \delta^D &\in \{-0.4, -0.133, 0.133, 0.4\}. \end{aligned}$$

The fine-grid check uses

$$\begin{aligned} \bar{w} &\in \{0.27, 0.575, 0.88\}, & \delta &\in \{-0.4, -0.2, 0, 0.2, 0.4\}, \\ \bar{p}^D &\in \{0.60, 1.10, 1.60\}, & \delta^D &\in \{-0.4, -0.2, 0, 0.2, 0.4\}. \end{aligned}$$

The intermediary response is computed over a three-parameter grid: retail base price, retail dynamic intensity, and routing price sensitivity.

S3. Artifact Map

Table 2: Artifact sources for main-paper quantities

Main-paper quantity	Artifact	Fields
Uncongested profit and utilization	artifacts/peak_shaving/20260618/peak_shaving_summary.json	uniform, dynamic.
Uncongested demand shift	artifacts/peak_shaving/20260618/peak_shaving_summary.json	P1_migration.rigid_shift, P1_migration.elastic_shift.
Elastic-population scan	artifacts/peak_shaving/20260618/peak_shaving_summary.json	elastic_sweep.
Congested uniform baseline	artifacts/peak_shaving/20260618/peak_shaving_congested_fp.json	uniform.system_profit, uniform.peak_util, uniform.min_qos_firm.
Dynamic coarse grid	artifacts/peak_shaving/20260618/peak_shaving_fp_dynamic_converged.json	system_profit, peak_util, min_qos, delta_A, delta_B.

Main-paper quantity	Artifact	Fields
Dynamic fine snapshot	artifacts/peak_shaving/ 20260618/peak_shaving_ fp_dynamic_converged_ fine.json	system_profit, peak_util, min_qos, delta_A, delta_B.
Fine-grid pure-snapshot re- gret	artifacts/peak_shaving/ 20260618/fp_dynamic_ ckpt_fine.json	round=40, maxreg=22.3029.
Coarse-grid regret	artifacts/peak_shaving/ 20260618/fp_dynamic_ ckpt.json	round=22, maxreg=4.9821.
Mechanism diagnostics	artifacts/peak_shaving/ 20260619/	peak_shaving_ diagnostics.json, CSV summaries, time profiles.
Mixed-strategy diagnostic	artifacts/peak_shaving/ 20260619_submission/ peak_shaving_mixed_ oracle.json	trace, expected_metrics, row_mix, col_mix.
Parameter stress tests	artifacts/peak_shaving/ 20260619_submission/ peak_shaving_parameter_ sweep_summary.json	QoS gains, peak-utilization re- ductions, profit signs.
SMPT diagnostics	artifacts/peak_shaving/ 20260619_smpt/	Strong baselines, ablations, phase grid, residuals, re- stricted re-solve.
Public price scale	artifacts/peak_shaving/ 20260619_submission/ public_api_price_anchor. json	Z.AI, DeepSeek, and OpenAI prices checked on 2026-06-19.
vLLM QoS anchor	artifacts/peak_shaving/ 20260619_submission/ vllm_qos_anchor_summary. json, vllm_qos_anchor_ points.csv	TTFT-SLA curves, fitted pa- rameters, and source CSVs.

S4. Submission-Strengthening Diagnostics

The high-regret fine-grid pure snapshot could be misread as an equilibrium if reported alone. The script `experiments/run_peak_shaving_mixed_oracle.py` uses a double-oracle diagnostic over the full 225-point fine grid. It starts from a small support, repeatedly adds best deviations, and solves an empirical fictitious-play profile on the restricted support. The current run obtains a 5×5 support in round 5. Full-grid maximum regret is 0.2025. The expected peak utilization is 0.7030, the minimum QoS is 0.9699, and system profit is 1733.13.

The script `experiments/run_peak_shaving_parameter_sweep.py` performs the local fixed-policy stress test. It evaluates nine perturbation scenarios covering capacity scale, price sensitivity, switching cost, and the QoS threshold. Both dynamic snapshots keep positive minimum-QoS gains and positive peak-utilization reductions in all nine scenarios. Profit is not stable: the dynamic fine snapshot is positive in only one scenario.

S5. SMPT Additional Diagnostics

The script `experiments/run_peak_shaving_smpt_experiments.py` adds diagnostics requested by the SMPT-oriented review. Outputs are stored under `artifacts/peak_shaving/20260619_smpt/` and `figures/peak_shaving_smpt/`. Table 3 lists the generated files.

Table 3: SMPT diagnostic artifact bundle

File	Contents
<code>peak_shaving_smpt_experiments.json</code>	Full bundle with metadata, baselines, ablations, phase grid, fixed-point residuals, and restricted resolved sensitivity.
<code>smpt_baselines.csv</code>	Static pricing, off-peak discount only, peak surcharge only, dynamic coarse/fine, equal routing, and admission-control diagnostic rows.
<code>smpt_ablations.csv</code>	Outside-option, direct-channel, capacity-homogeneity, time-flexibility, and equal-routing ablations.
<code>smpt_phase_grid.csv</code>	Fixed-policy grid over capacity scale and price-sensitivity scale.
<code>smpt_fixed_point_residuals.csv</code>	Iterations and final residuals for the main baseline and dynamic-policy fixed points.
<code>smpt_vv_damping_initial.csv</code>	Initial-condition and damping-factor audit for uniform, dynamic coarse, and dynamic fine fixed points.
<code>smpt_resolved_sensitivity.csv</code>	Restricted local candidate re-solve for five selected perturbation scenarios.

The strongest new baseline comparison is not a new equilibrium proof. It shows that one-sided price shapes improve QoS but do not match the full dynamic coarse policy: off-peak-only discounting reaches minimum QoS 0.833, peak-only surcharge reaches 0.921, and dynamic coarse reaches 0.968. Equal routing with the same dynamic coarse prices reaches 0.881, showing that routing and price timing both contribute. The admission-control diagnostic indicates that matching the dynamic coarse peak-utilization target with static pricing would require a minimum admitted fraction of 0.902 in the tightest period.

The fixed-point residual check reports final residuals below 10^{-9} for the static baseline, off-peak-only, peak-only, dynamic coarse, and dynamic fine rows. The damping and initialization audit evaluates three initial QoS levels and three damping factors for uniform, dynamic coarse, and dynamic fine. It converges in 24 of 27 rows. Dynamic coarse and dynamic fine converge in all 18 rows; the three non-converged rows are the static uniform baseline with damping 0.5. The fixed-policy phase grid has positive QoS gain and peak-utilization reduction in all 25 capacity-elasticity grid points. The restricted local candidate re-solve keeps positive QoS gain in all five selected scenarios; profit is positive in three and negative in two, preserving the profit-boundary conclusion.

S6. Price and QoS Anchors

The public price anchor is stored in `artifacts/peak_shaving/20260619_submission/public_api_price_anchor.json`. It is used only to check order of magnitude. It is not an estimate of demand elasticity.

The controlled vLLM QoS-shape anchor is produced by `experiments/build_peak_shaving_measurement_anchor.py`. Inputs are `artifacts/vllm-study/20260531-190126/controlled_`

aggregate.csv and artifacts/vllm-study-qwen25-3b/20260531-214710/controlled_aggregate.csv. Both scans use vLLM 0.22.0 and an NVIDIA GeForce RTX 4090. QoS is the attainment rate for a 0.5-second time-to-first-token SLA. Table 4 reports the fitted threshold-type QoS parameters.

Table 4: Controlled vLLM single-GPU QoS-shape anchor

Profile	Healthy conc.	$\bar{u}$	κ	RMSE	Drop conc.	Min QoS
Qwen2.5-0.5B	224	1.000	0.517	0.068	384	0.500
Qwen2.5-3B	224	1.058	0.942	6.3×10^{-10}	384	0.667

S7. Reproduction Commands

Run the following commands in /root/paper_code/0427_tokenrl/paper_token_cross_survey. The public repository is https://github.com/cccht/paper_token_price; a formal submission should cite a frozen archival release when available.

```
uv run --with-requirements requirements.txt \
    python experiments/build_peak_shaving_diagnostics.py
uv run --with-requirements requirements.txt \
    python experiments/run_peak_shaving_parameter_sweep.py
uv run --with-requirements requirements.txt \
    python experiments/run_peak_shaving_mixed_oracle.py
uv run --with-requirements requirements.txt \
    python experiments/run_peak_shaving_smpt_experiments.py
uv run --with-requirements requirements.txt \
    python experiments/run_peak_shaving_smpt_vv_audit.py
uv run --with-requirements requirements.txt \
    python experiments/build_peak_shaving_measurement_anchor.py
```

Compile the English manuscript and supplement with:

```
xelatex -interaction=nonstopmode -halt-on-error \
    peak_shaving_dynamic_pricing_sci_en_2026-06-19.tex
bibtex peak_shaving_dynamic_pricing_sci_en_2026-06-19
xelatex -interaction=nonstopmode -halt-on-error \
    peak_shaving_dynamic_pricing_sci_en_2026-06-19.tex
xelatex -interaction=nonstopmode -halt-on-error \
    peak_shaving_dynamic_pricing_sci_en_2026-06-19.tex
xelatex -interaction=nonstopmode -halt-on-error \
    peak_shaving_dynamic_pricing_sci_en_supplement_2026-06-19.tex
```

S8. Known Boundaries

The fine-grid pure snapshot does not reach the regret < 5 criterion within 40 rounds. Its profit, dynamic intensity, and demand-centroid values should be read as high-regret snapshot diagnostics. The mixed-strategy diagnostic lowers regret on the finite 225-point grid, but it is not a continuous-strategy equilibrium proof.

The SMPT diagnostics strengthen the baseline and sensitivity evidence but do not remove the calibration boundary. The phase grid is still fixed-policy evidence, and the re-solved sensitivity uses a restricted local candidate set rather than a full continuous-strategy search. The paper does not calibrate price sensitivity, switching cost, or outside-option utility from real request traces.

Public API prices provide only a price-scale anchor. The vLLM scans provide only a single-GPU, small-model, short-generation QoS-shape anchor. Production use would require traces, multi-GPU measurements, long-context workloads, user behavior calibration, and online evaluation.